Callzo Engineering Assessment

Role: Software Engineer Intern

Deadline: 10 days from when this assignment is sent to you

Format: Take-home build

AI tools: Feel free to use AI, but make sure you understand every line of code and the decision behind why it was written that way. We will ask you to defend and change your design live.

You must complete both tasks below. One tests backend thinking, the other tests UI thinking, and we want to see both before we meet.

Task 1: Backend, Usage Metering Service

Build a service that receives call events from a telephony provider, stores them, computes usage per account, enforces a monthly cap, and exposes usage through an authenticated API.

Stack: Anything you're comfortable in (Python FastAPI recommended) **Database:** Postgres

Requirements

- Accept webhooks with signature verification. Reject requests with invalid signatures before processing the payload.
- Handle duplicate, late, and out-of-order events. The same event may arrive more than once, and a "call ended" event may arrive before "call started".
- Store usage in Postgres with a schema you can justify. Money or cost values must use integer units. Use transactions where correctness depends on them.
- Enforce a monthly cap. Decide whether it is soft or hard, and document the behavior at the boundary.
- Include a background job for retries, plus a reconciliation step that can recompute usage from raw events.
- Authenticate the API, validate all input, and keep secrets out of the codebase.
- Add structured logs and at least one metric or health check.
- Must run with `docker compose up`. Include a Dockerfile and compose file covering the API and Postgres.

Provided (See Task1_spec.md file attached): A webhook simulator (Docker image) and a spec describing the event schema, signature scheme, and shared secret.

Deliverables

- Working code with tests
- A README covering setup, design decisions, failure behavior, and what you would do next with more time

---

Task 2: UI, Usage Admin Console

Build an admin console for viewing and editing account usage. The data comes from a mock API that we provide as a Docker image.

Stack: React + Tailwind + any chart library of your choice. Do not use any other library beyond these.

Requirements

- A data table with server-side search, sorting, and filters. Row selection stays stable across pages.
- A date-range filter that updates both the table and a summary chart without a full page reload.
- A 4-step account settings form. It autosaves drafts, lets users go back to earlier steps, and shows server-side validation errors on the step they belong to, even if later steps are already filled.
- Loading, empty, error, and partial-failure states on every async view. Failed saves must roll back cleanly.
- Every action works with the keyboard. Use proper labels and focus management.
- Usable at 375px width.
- Data refreshes periodically. The UI must not jump or lose the user's selection during a refresh.
- Must run with `docker compose up`, serving both the app and the mock API.

Provided (See Task2_Spec.md) : A mock API (Docker image) and a spec describing every endpoint, the schema, and two deliberate behaviors you'll need to handle: an endpoint that fails randomly, and a validation error that only appears on step 2 after step 4 is filled.

Deliverables

- Working React app
- A short note on where your state lives (server, client, or URL) and why
- A short note on what you cut and why

How We Evaluate

We score each task on these areas, 1 to 5 each:

1. **Clarifies before building:** Identifies ambiguous requirements and states assumptions.
2. **Handles failure cases:** Covers the failure cases in the requirements, not just the happy path.
3. **Explains trade-offs:** Names what was given up, not only what was chosen.
4. **Holds up under change:** Adapts the design when requirements change during the walkthrough.
5. **Shows taste and simplicity:** Solves the problem with the least complexity that works. We're not impressed by clever code; we want code a teammate can read and change in 6 months.
6. **Code quality:** Clear naming, small functions, consistent style, no dead code or leftover debug output.
7. **Documentation:** The README lets a stranger run the project and understand the key decisions without asking you anything.
8. **Runs cleanly in Docker:** Starts with one command, with no manual setup steps left out of the instructions.

Submission

- Put both tasks in a **single Git repository**, with one folder per task.
- If the repository is private, **invite the user `Meetrj`** before you submit, and make sure the invite is sent.
- Share the repository link, along with:

1. Your README for each task
2. Confirmation that `docker compose up` runs each project